01.

Unit 1

Welcome to Algorithm Design

- Explore the foundations of **problem-solving** in computer science

Modern Approach

- Clean, minimal visuals with tech-inspired themes

Course Focus

- Learn to build **efficient** and **scalable** solutions

01.

Algorithm Design Basics

What is an Algorithm?

- A step-by-step process to solve a problem
- Clear instructions for tasks like making tea

Characteristics of Algorithms

- Requires **input** and produces **output**
- Must be **definite**, **finite**, and **effective**

Why Algorithms Matter

- Drive **efficiency** and **scalability**
- Enable real-world problem solving

02.

Agenda

1. Introduction to Algorithms
2. Algorithm Types and Problem Solving
3. Complexity Concepts
4. Summary and Practice

03.

Introduction to Algorithms

Welcome to Algorithm Design

What is Algorithm Design?

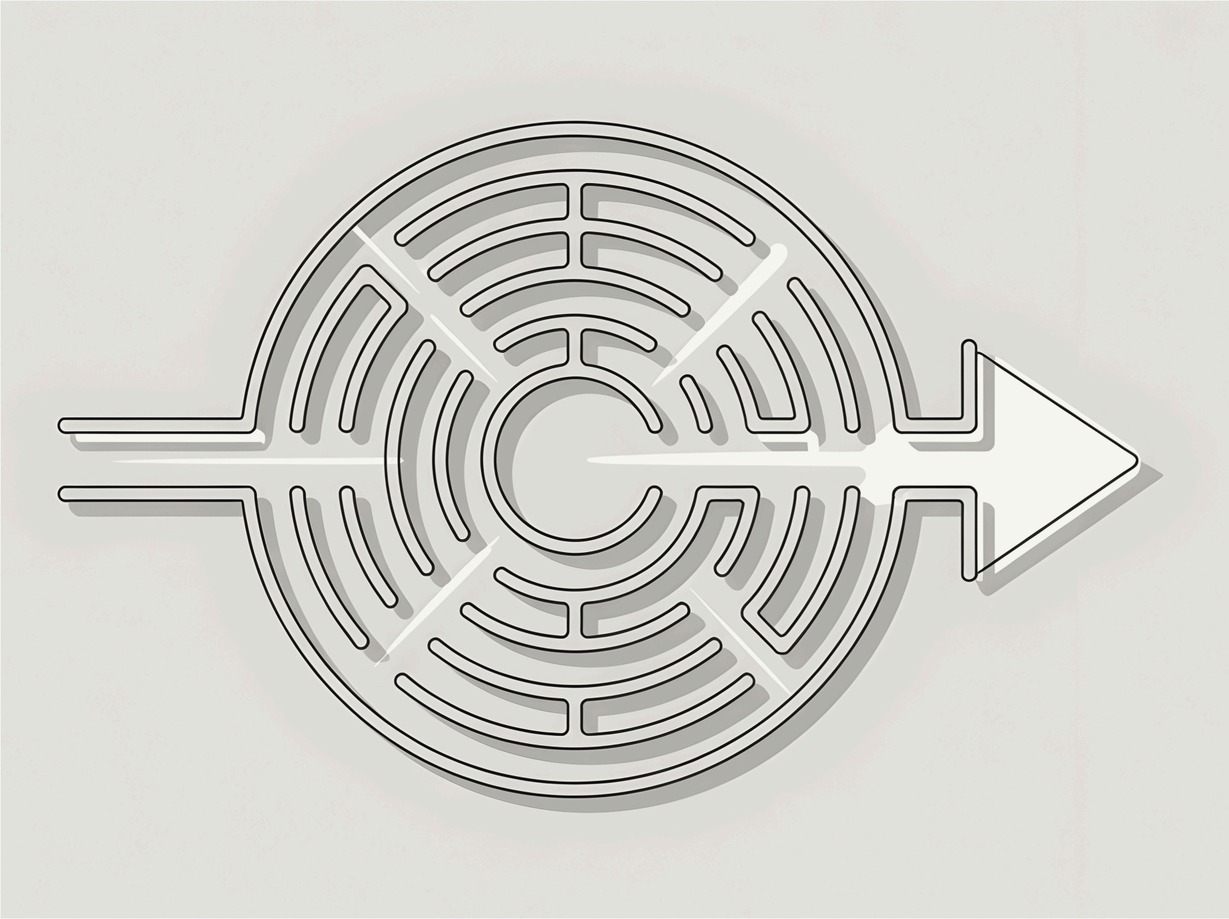
Foundation for solving complex problems efficiently

Real-World Impact

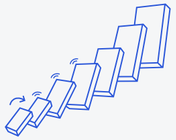
Algorithms power everything from search engines to smart devices

Modern Tech Inspiration

Visuals inspired by AI, circuits, and abstract shapes set the tone

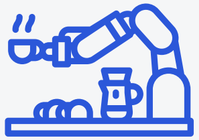


What Is an Algorithm?



Defining an Algorithm

- A set of **step-by-step instructions** to solve a problem
- Ensures a clear path from **input** to **output**



Real-Life Example: Making Tea

- Boil water
- Add tea leaves
- Steep for a few minutes
- Pour into cup and enjoy

Key Traits of Algorithms

Clear Inputs and Outputs

Every algorithm starts with defined inputs and produces specific outputs.

Definiteness: Step-by-Step Precision

Instructions must be unambiguous and precisely stated for each step.

Finiteness: Guaranteed Completion

Algorithms must finish after a limited number of steps—no endless loops.

Effectiveness: Practical Execution

Each step should be simple enough to be carried out in practice.

Why Algorithms Matter

Boosting Efficiency

- Algorithms streamline tasks for **faster results**

Scaling Solutions

- Handle larger data and complex problems with ease

Empowering Problem Solving

- Transform challenges into **actionable steps**

Real-World Impact

- Algorithms drive innovations in tech, healthcare, and finance

08.

Algorithm Types and Problem Solving

Exploring Algorithm Types

Brute Force Approach

- Tries all possible solutions to find the answer
- Simple but often inefficient for large problems

Divide & Conquer Strategy

- Breaks problem into smaller subproblems
- Solves each part recursively and combines results

Greedy Algorithms

- Makes the best choice at each step
- Fast and simple, but may not always find optimal solution

Dynamic Programming

- Stores solutions to subproblems to avoid repetition
- Efficient for problems with overlapping subproblems

Solving a Sample Problem

Step-by-Step Array Search

- Start with the first element as the **current largest**
- Compare each element to the current largest
- Update the largest when a bigger value is found
- Finish with the largest value after checking all elements

11.

Complexity Concepts

Understanding Time Complexity

Defining Time Complexity

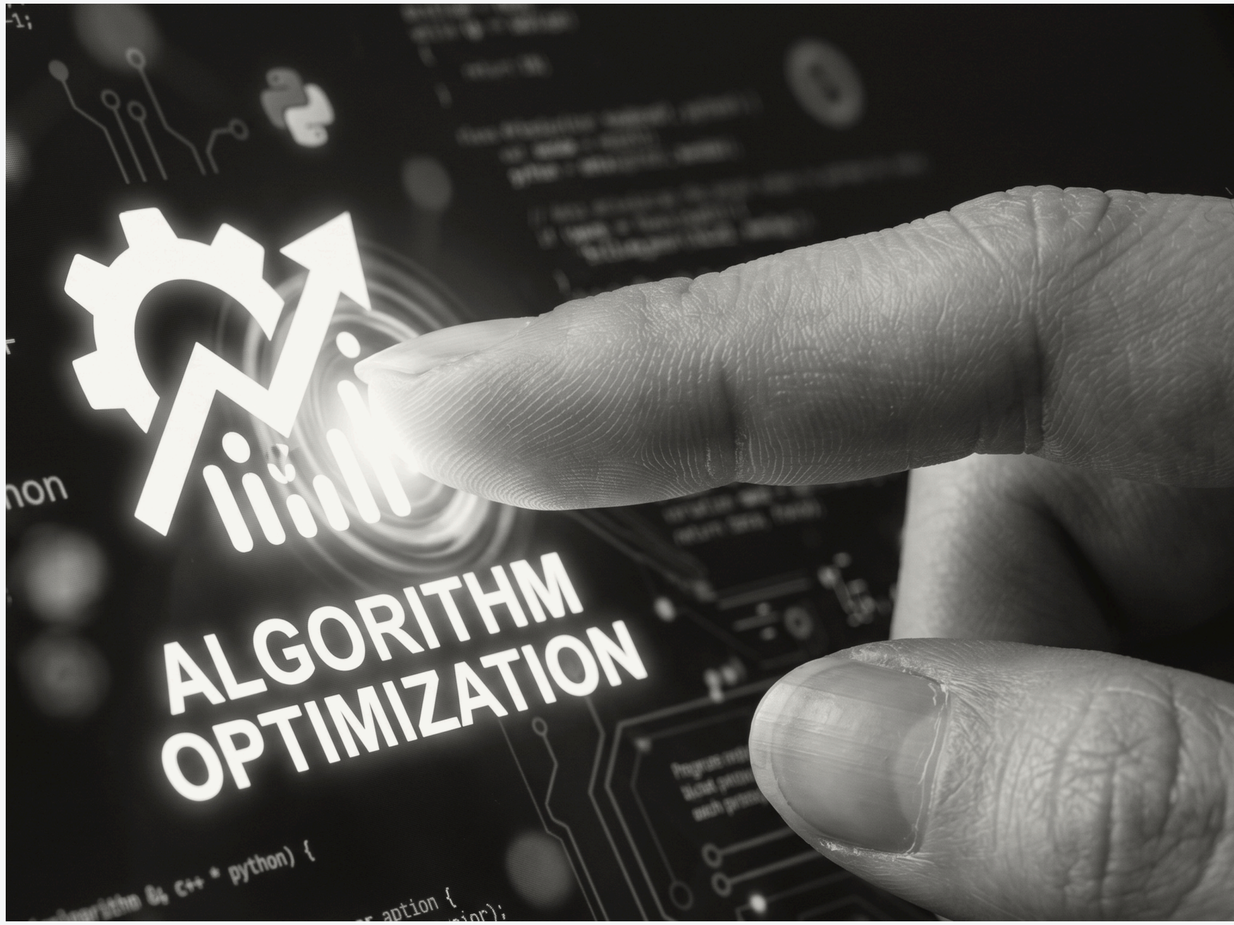
- Measures how long an algorithm takes to run as input size grows.
- Helps compare efficiency of different solutions.

Big-O Notation Basics

- Describes the upper bound of an algorithm's running time.
- Focuses on the most significant factors as input increases.

Why It Matters

- Guides us to build **scalable** and **efficient** algorithms.



Comparing Common Complexities

Constant Time: O(1)

Operation takes same time regardless of input size

Linear Time: O(n)

Time grows directly with input size

Logarithmic Time: O(log n)

Efficient for large data; halves input each step

Quadratic Time: $O(n^2)$

Time increases rapidly as input grows

Visual Complexity Comparison

Graph shows how each complexity scales with input

Space Complexity Basics

Understanding Space Complexity

- Measures how much **memory** an algorithm uses during execution
- Helps predict scalability and efficiency

Simple Example: Array Search

- Finding the largest number in an array only needs a few variables
- Space complexity is **O(1)** (constant), regardless of array size

Why It Matters

- Efficient memory use is crucial for large-scale applications

15.

Summary and Practice

Key Takeaways

Core Algorithm Concepts

- Algorithms solve problems efficiently with clear steps.
- Key characteristics: input, output, definiteness, finiteness, effectiveness.

Types & Approaches

- Brute Force, Divide & Conquer, Greedy, Dynamic Programming each offer unique strategies.

Complexity Matters

- Understanding **time complexity** and **space complexity** is essential for scalable solutions.

Practice Makes Perfect

Try Algorithm Challenges

- Practice finding the **largest number** in an array.
- Write steps to **sort a list** of numbers.

Apply Concepts to Real Problems

- Design an algorithm to check if a word is a **palindrome**.

Build Confidence Through Practice

- Hands-on exercises reinforce learning and boost problem-solving skills.

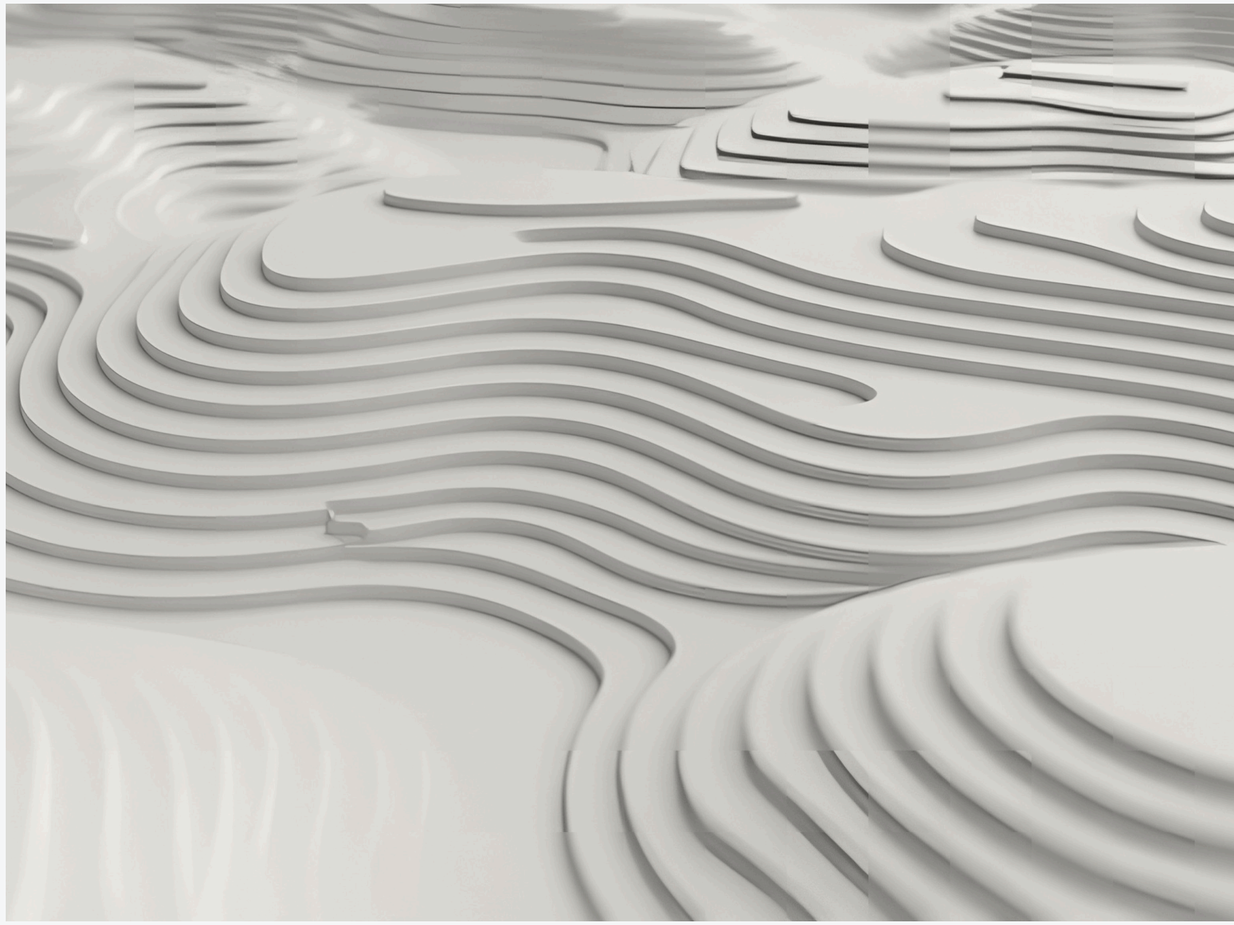
Thank You and Next Steps

Embrace Algorithmic Thinking

- Apply your skills to solve **real-world challenges**

Keep Exploring

- Experiment with new problems and approaches

Collaborate and Innovate

- Work with peers to build **smarter solutions**